Software Requirements Specification for LingoLyrics: A Language Learning System Based on Songs

Team #21, LingoLyrics
Rhea Gokhale
Aansh Kotian
Omar Bakr
Raymond Ma

November 5, 2026

Contents

1 Purpose of the Project **3**
- 1.1 User Business . . . 3
- 1.2 Goals of the Project . . . 3

2 Stakeholders **4**
- 2.1 Client . . . 4
- 2.2 Customer . . . 4
- 2.3 Other Stakeholders . . . 4
- 2.4 Hands-On Users of the Project . . . 4
- 2.5 Personas . . . 5
- 2.6 Priorities Assigned to Users . . . 6
- 2.7 User Participation . . . 6
- 2.8 Maintenance Users and Service Technicians . . . 7

3 Mandated Constraints **7**
- 3.1 Solution Constraints . . . 7
- 3.2 Implementation Environment of the Current System . . . 7
- 3.3 Partner or Collaborative Applications . . . 8
- 3.4 Off-the-Shelf Software . . . 8
- 3.5 Anticipated Workplace Environment . . . 8
- 3.6 Schedule Constraints . . . 8
- 3.7 Budget Constraints . . . 9
- 3.8 Enterprise Constraints . . . 9

4 Naming Conventions and Terminology **9**
- 4.1 Glossary of All Terms, Including Acronyms, Used by Stakeholders involved in the Project . . . 9
- 4.2 Abbreviations and Acronyms . . . 10

5 Relevant Facts And Assumptions **11**
- 5.1 Relevant Facts . . . 11
- 5.2 Business Rules . . . 12
- 5.3 Assumptions . . . 12

6 The Scope of the Work **13**
- 6.1 The Current Situation . . . 13
- 6.2 The Context of the Work . . . 13

6.3 Work Partitioning 14
6.4 Specifying a Business Use Case (BUC) 15

7 Business Data Model and Data Dictionary 15
7.1 Business Data Model 15
7.2 Data Dictionary 16

8 The Scope of the Product 17
8.1 Product Boundary 17
8.2 Product Use Case Table 17
8.3 Individual Product Use Cases (PUC's) 17

9 Functional Requirements 18
9.1 Functional Requirements 18

10 Formal Specification 22
10.1 Song Map 22
10.2 Active Line and Synchronization 23
10.3 Playback Modes 23

11 Look and Feel Requirements 24
11.1 Appearance Requirements 24
11.2 Style Requirements 25

12 Usability and Humanity Requirements 25
12.1 Ease of Use Requirements 25
12.2 Personalization and Internationalization Requirements 25
12.3 Learning Requirements 26
12.4 Understandability and Politeness Requirements 26
12.5 Accessibility Requirements 26

13 Performance Requirements 27
13.1 Speed and Latency Requirements 27
13.2 Safety-Critical Requirements 27
13.3 Precision or Accuracy Requirements 30
13.4 Robustness or Fault-Tolerance Requirements 31
13.5 Capacity Requirements 31
13.6 Scalability or Extensibility Requirements 31
13.7 Longevity Requirements 32

14 Operational and Environmental Requirements **32**
14.1 Expected Physical Environment 32
14.2 Wider Environment Requirements 32
14.3 Requirements for Interfacing with Adjacent Systems 32
14.4 Productization Requirements 33
14.5 Release Requirements 33

15 Maintainability and Support Requirements **33**
15.1 Maintenance Requirements 33
15.2 Supportability Requirements 34
15.3 Adaptability Requirements 34

16 Security Requirements **34**
16.1 Access Requirements 34
16.2 Integrity Requirements 35
16.3 Privacy Requirements 38
16.4 Audit Requirements 39
16.5 Immunity Requirements 39

17 Cultural Requirements **40**
17.1 Cultural Requirements 40

18 Compliance Requirements **41**
18.1 Legal Requirements 41
18.2 Standards Compliance Requirements 42

19 Open Issues **43**

20 Off-the-Shelf Solutions **43**
20.1 Ready Made Products 43
20.2 Reusable Components 44
20.3 Products That Can Be Copied 44

21 New Problems **44**
21.1 Effects on the Current Environment 44
21.2 Effects on the Installed Systems 45
21.3 Potential User Problems 45
21.4 Limitations in the Anticipated Implementation Environment That May Inhibit the New Product 45

21.5 Follow-Up Problems 45

22 Tasks **46**
22.1 Project Planning 46
22.2 Planning of the Development Phases 46
22.3 Requirement Priorities and Phase In Plan 46

23 Migration to the New Product **47**
23.1 Requirements for Migration to the New Product 47
23.2 Data That Has to be Modified or Translated for the New System 48

24 Costs **48**

25 User Documentation and Training **48**
25.1 User Documentation Requirements 48
25.2 Training Requirements 48

26 Waiting Room **50**

27 Ideas for Solution **50**

28 Likely and Unlikely Changes **51**
28.1 Likely Changes 51
28.2 Unlikely Changes 51

A Symbolic Constants **52**

B Traceability **54**
B.1 Goals to Requirements 54
B.2 Use Cases to Requirements 54
B.3 Hazards to Requirements 54

References **56**

C Appendix: Generative AI Use Disclosure **58**
C.1 AI Tools Used 58
C.2 How and Where Used 58
C.3 Verification of AI-Generated Content 59
C.4 Limitations 60

D Appendix: Reflection 61

List of Tables

1 User characteristics 5
3 Abbreviations and acronyms 11
4 Business events 14
6 Learner app playback mode transitions 24
7 Requirement priorities and phase in dates 47
8 Possible costs 49
10 Goals and the requirements that refine them 54
11 Product use cases and functional requirements 54
12 Hazard Analysis FMEA rows and mitigating requirements . . 55
13 AI tools used for this document 58

List of Figures

1 Context of the work. Each box is a person or system, and each arrow shows data flowing in the direction it points, labelled with what flows. The two way arrow means the learner both sends selections and receives the lesson view. 14

Revision History

Date	Version	Notes
October 5th, 2026	0.1	Initial draft alignment with project planning and requirements sources.
October 18, 2026	1.1	Filled the remaining Volere sections. Added fit criteria, rationale, and traces to each requirement. Added the safety and security requirements from the Hazard Analysis. Added symbolic constants, a formal specification, likely and unlikely changes, and traceability tables.
October 18, 2026	1.2	Addressed review comments: added HS8 (fade in) and HS9 (loudness reporting), narrowed HS3, IN5, IN7, and AU1 to the capstone scope, revised AC2 and PR1, replaced the privacy legislation statement, corrected the product comparisons, added references, and completed the verification of AI-generated content.
October 19, 2026	1.3	Matched the Hazard Analysis revision: extended IM2 to reject paths that leave the map package (H4.5), added IM3 for third party libraries (H7.3), and added both hazards to the traceability table.
October 19, 2026	1.4	Revised against the grading rubric: added the intended audience, notation and conventions, the list of acronyms, user characteristics, a third persona, the context diagram legend, links to every requirement and *Referenced by* lines, the phase in plan with priorities and dates, the reflection answers, and tool versions in the AI disclosure.

About This Document

This document specifies the requirements for LingoLyrics, a language learning product based on songs. It is written for the development team, who will design, build, and test the product from it, for new team members who need to get up to speed, and for the supervisor, teaching staff, and peer reviewers who assess it. A reader outside the team should be able to design and test the product from this document, the Problem Statement, and the Hazard Analysis alone.

Template and Changes

The document follows the Volere template [1], with the following changes:

- The *Risks* section is removed because project risks are covered by the Development Plan (Proof of Concept (PoC) risks R1 to R3) and hazards are covered by the Hazard Analysis.
- Section 10 (*Formal Specification*) is added. It defines a valid song map, the line that must be highlighted at a given playback time, and the learner app's playback modes.
- Section 28 (*Likely and Unlikely Changes*) is added.
- Appendix A (*Symbolic Constants*) and Appendix B (*Traceability*) are added.
- Each requirement is written as a statement followed by its rationale, fit criterion, and the goals, use cases, or hazards it traces to. Numeric values are written as symbolic constants such as `SYNC_TOL`. Their values and the reason for each value are in Appendix A.
- Section 4.2 (*Abbreviations and Acronyms*) is added after the glossary.
- Section 22.3 (*Requirement Priorities and Phase In Plan*) is added under *Tasks*.

Notation and Conventions

- **Requirement identifiers.** Every requirement has a unique identifier made of a prefix and a number. The prefixes are MC (mandated constraint), FR (functional), LF (look and feel), UH (usability and humanity), SL (speed and latency), PA (precision or accuracy), RB (robustness), CP (capacity), SC (scalability), LO (longevity), OE (operational and environmental), MS (maintainability and support), CU (cultural), and ST (standards). The prefixes HS (health and safety), AC (access), IN (integrity), PR (privacy), AU (audit), IM (immunity), and LG (legal) are shared with the Hazard Analysis, so each safety or security requirement has the same identifier in both documents. Goals are G1 to G6, assumptions A1 to A4, business rules BR1 to BR3, likely changes LC, and unlikely changes UC. Rows of the Failure Mode and Effects Analysis (FMEA) in the Hazard Analysis are written H1.1 to H7.3.

- **Requirement entries.** Each requirement is a statement using *shall*, followed by its *Rationale*, a *Fit criterion* that says how it will be tested, *Traces to* (the goals, use cases, hazards, and requirements it depends on or refines), and *Referenced by* (the requirements that trace to it). Every requirement identifier in a trace is a link to that requirement, so each dependency can be followed in both directions.

- **Symbolic constants** are written in `TYPEWRITER` type and defined only in Appendix A. Field names of the song map are also in `typewriter` type and are defined in the data dictionary (Section 7).

- **Mathematical notation** in Section 10: $\langle \ldots \rangle$ is a sequence, $[1..n]$ is the integers from 1 to n, $\mathbb{R}^+$ is the positive real numbers, ε is the empty string, $\perp$ means *no value*, and $\forall$, $\wedge$, $\neg$, and $\equiv$ are *for all*, *and*, *not*, and *is defined as*. Times are in seconds from the start of the audio.

- **Terms** with a specific meaning in this document are defined in the glossary (Section 4.1), and acronyms are written out on first use and listed in Section 4.2.

- **Diagrams** have a legend that explains each shape and arrow.

1 Purpose of the Project

1.1 User Business

Language learners who study on their own at early intermediate to intermediate levels often use songs to hear vocabulary, idioms, and cultural references as native speakers use them. Media players and lyrics websites do not support studying this material. When a learner meets an unfamiliar word, they have to pause the audio, look the word up in another tool, and copy it into a flashcard application by hand. This interrupts listening and separates the word from the lyric that gives it its meaning.

LingoLyrics combines synchronized lyrics, translations of each line, transliterations for scripts other than Latin, contextual annotations, and vocabulary export to Anki in one workflow.

1.2 Goals of the Project

The goals below are taken from the Problem Statement and Goals document. Each goal's measure is refined into the requirements listed in Table 10.

G1 **Synchronized Lesson Playback:** highlight each lyric line within `SYNC_TOL` of its mapped timestamp for at least `SYNC_PASS` of line transitions in the acceptance suite.

G2 **Vocabulary Support in Context:** show the translation and contextual annotation of every annotated target when it is selected.

G3 **Interoperability between Creators and Learners:** a song map created by the editor validates against one shared format and opens in the learner app without manual changes.

G4 **Practical Lesson Creation:** at least `CREATE_PASS` test creators produce a complete map for a test track of at most `TEST_TRACK_LINES` lines within `CREATE_TIME`, without editing the map file by hand.

G5 **Support for Different Writing Systems:** display lyrics correctly for `LANG_COUNT` languages, at least one with a writing system other than Latin, with word divisions and transliterations that match the song map.

G6 **Vocabulary Retention Handoff:** sending a selected item creates an Anki note with correct *Term*, *Translation*, and *Context* fields.

2 Stakeholders

2.1 Client

The project has no external client. The project supervisor, Dr. Angela Zavaleta Bernuy, acts as the client: the supervisor reviews the scope and goals and accepts the deliverables at each milestone.

2.2 Customer

The customers are language learners at early intermediate to intermediate levels who study on their own who want to study vocabulary from songs, and song map creators who want a tool for preparing synchronized song lessons.

2.3 Other Stakeholders

- The LingoLyrics capstone development team (Omar Bakr, Raymond Ma, Rhea Gokhale, Aansh Kotian).
- The course instructor and teaching assistants, who evaluate the deliverables.
- Anki users and the maintainers of Anki, AnkiDroid, and AnkiConnect, whose software receives the exported notes.
- Copyright holders of songs and lyrics, whose rights limit what content can be distributed.

2.4 Hands-On Users of the Project

- **Learners:** use the mobile app on an iOS or Android phone, usually with headphones, to study song lessons. They are assumed to be comfortable with common phone apps and to have at least beginner knowledge of the target language. They are not assumed to know linguistic terms such as *segmentation*.

- **Creators:** language tutors or advanced learners who use the web editor on a laptop or desktop computer to time lyrics and write translations and annotations. They are assumed to read the target language well and to be able to work with audio and text files.

Table 1 summarizes the characteristics of each user group that affect the requirements.

Table 1: User characteristics

Characteristic	**Learners**	**Creators**
Age and setting	Adults and older teenagers studying on their own	Adult tutors, teachers, and advanced learners
Language level	Early intermediate to intermediate in the target language	Reads the target language well, often a native speaker
Device	iOS or Android phone, usually with headphones	Laptop or desktop computer with a current browser
Technical skill	Comfortable with common phone apps. May already use Anki. No knowledge of linguistic terms or file formats	Comfortable with audio and text files. No programming or JSON knowledge assumed
Session length	Short sessions of 5 to 15 minutes, often in transit	Longer sessions at a desk, 30 to 60 minutes per song
Accessibility needs	May have low vision, colour vision deficiency, or photosensitivity, and may use a screen reader	May have the same needs, and may work with Bluetooth headphones that add audio latency
Motivation	Enjoys music and wants to understand it, but gives up when lookups break the flow	Wants to share lessons without preparing the same context again by hand
Requirements driven	UH1, UH3, UH5, UH7, HS3 to HS7, RB2, OE1	FR9 to FR15, UH2, IN6, IN7, MS3

2.5 Personas

Each persona describes a user's goal, what frustrates them today, and how they feel about it, and names the requirements that respond to them.

- **Kenji (learner):** a Japanese learner at early intermediate level who listens to Japanese pop music on the train but cannot read kanji at singing speed. Today he pauses the song, copies a word into a dictionary, and types it into Anki, and by then the line has passed. He feels the song stops being fun as soon as it becomes study. He needs to follow the romanized lyrics (FR6), tap a word without stopping the music (FR7), and send it to Anki in one tap with the lyric line attached (FR8), knowing it was saved (IN8).

- **Elena (creator):** a language tutor who prepares song lessons for her students. Today she builds them in slides with no timing, and every student has to search the song again. She worries about spending an evening on a lesson and losing it, and about using songs she does not have the right to share. She needs fast timing by tapping along (FR10), autosave and undo (IN6), checks that catch mistakes before export (FR14), and licence fields that remind her what she may share (LG1).

- **Amara (learner with low vision):** studies Japanese with large system text and sometimes uses VoiceOver. Most lyric apps mark the current line only with a colour change she cannot see well, and she feels excluded by apps that ignore her text size. She needs the active line marked by more than colour and text that follows her settings (HS5), labelled controls (UH7), and sufficient contrast (ST1).

The value of LingoLyrics compared with the products that learners use today is described in Section 20.1.

2.6 Priorities Assigned to Users

Learners have first priority because they are the users of the lessons: playback timing, word selection, and Anki export are developed first. Creators have second priority. The editor is needed to produce lessons but is used by fewer people.

2.7 User Participation

Representative learners and creators will take part in the interviews for the Fall custom document and in the usability testing for the Winter custom

document (Usability Report). The usability sessions measure the fit criteria of UH1 to UH4 and G4.

2.8 Maintenance Users and Service Technicians

The development team maintains the product. There are no separate service technicians.

3 Mandated Constraints

3.1 Solution Constraints

The requirements in this document say *what* the product must do and avoid choosing a design. The few design decisions that are already fixed are recorded here as constraints, each with its reason. MC1 and MC2 were agreed in the Development Plan and reviewed by the supervisor before this document was written. MC3 follows from goal G3.

MC1 All application code and the shared song map types shall be written in TypeScript.
Rationale: one language for the app, editor, and shared code lets all three use the same song map types and validator.

MC2 The learner app shall be built with React Native and Expo. The editor shall be built with React.
Rationale: one code base covers iOS and Android, and the editor can reuse the app's TypeScript code (Development Plan, Expected Technology).

MC3 Lessons shall be stored and exchanged as song maps that conform to one versioned JSON Schema (a schema for JavaScript Object Notation files).
Rationale: required by goal G3.

3.2 Implementation Environment of the Current System

There is no current system. The learner app will run on iOS and Android phones that meet the minimum operating system versions of the Expo

software development kit (SDK) release used. The editor will run in current desktop versions of Chrome, Firefox, Safari, and Edge.

3.3 Partner or Collaborative Applications

- **Anki, AnkiDroid, and AnkiConnect:** receive vocabulary notes from the learner app (Section 14.3).

3.4 Off-the-Shelf Software

- **Expo Audio and Howler.js:** audio playback with position reporting in the app and the editor.
- **kuromoji.js:** Japanese segmentation and readings in JavaScript.
- **A JSON Schema validator library:** validates song maps in the editor, the app, and CI.
- **Jest and ESLint:** testing and code quality tools.

3.5 Anticipated Workplace Environment

Learners use the app at home, in transit, or while walking, usually with headphones and in short sessions. Creators work at a desk on a laptop or desktop computer, with headphones or speakers.

3.6 Schedule Constraints

The project follows the capstone deadlines in the Development Plan:

- SRS and Hazard Analysis: October 19, 2026
- Proof of Concept demonstration: November 23 to December 4, 2026
- Revision 0 demonstration: February 1 to 12, 2027
- Final demonstration (Revision 1): March 15 to 22, 2027
- Final documentation package: March 31, 2027

3.7 Budget Constraints

No budget is allocated. The project uses free and open source tools and team members' own devices. Costs that may arise are listed in Section 24.

3.8 Enterprise Constraints

Code and documentation are released under the MIT License and hosted publicly on GitHub.

4 Naming Conventions and Terminology

4.1 Glossary of All Terms, Including Acronyms, Used by Stakeholders involved in the Project

Term	Meaning
Acceptance suite	The fixed set of `LANG_COUNT` song maps, one per supported language, used to evaluate the goals. It is fixed before implementation testing begins.
Annotated target	A word or line that has both a translation and a contextual annotation.
Anki	A spaced repetition flashcard application. AnkiDroid is its Android version. AnkiConnect is an extension that lets other programs add notes to desktop Anki.
Contextual annotation	An explanation of a word, phrase, or line that gives its meaning in the song.
Creator	A person who prepares song maps with the editor.
Editor	The song map editor, which runs in a web browser.
Explicit content	Lyrics containing profanity, slurs, or sexual or violent content.
JSON Schema	A language for describing and validating the structure of JavaScript Object Notation (JSON) data [10].

Term	Meaning
LUFS	Loudness units relative to full scale, the unit of integrated loudness defined in ITU-R BS.1770 [12]. LU is a difference in loudness.
Learner	A person who studies song lessons with the learner app.
Learner app	The mobile application used by learners.
Lesson	A song map and its audio, as opened in the learner app.
PoC	Proof of Concept.
Romanization	Transliteration into the Latin alphabet (for Japanese, *rōmaji*).
Segmentation	Splitting a line of text into words, needed for languages such as Japanese that do not put spaces between words.
Song map	A file that connects an audio track with its timed lyric lines, segmentation, transliterations, translations, annotations, and metadata. Its structure is defined in Section 7.
Song map package	A folder or archive containing one song map and the audio file it refers to.
SRS	Software Requirements Specification.
Test creator	A participant given the same audio, lyric, and annotation material for evaluating G4.
Transliteration	Writing a word in another script so that a learner can pronounce it.
Verified	Describes a segmentation or reading that a creator has confirmed in the editor.

4.2 Abbreviations and Acronyms

Table 3 gives the full form of every acronym in this document and links to where it is defined or first explained.

Table 3: Abbreviations and acronyms

Acronym	Full form	Defined in
API	Application programming interface	Section 5.1
BR	Business rule	Section 5.2
BUC	Business use case	Section 6.4
CI	Continuous integration, the automated checks run on every pull request	Section 9.1
FMEA	Failure Mode and Effects Analysis	Hazard Analysis
JSON	JavaScript Object Notation	Section 4.1
LC, UC	Likely change, unlikely change	Section 28
LRC	Lyric file format with timed lines	Section 20.3
LU, LUFS	Loudness unit, loudness units relative to full scale	Section 4.1
MIT	The MIT License, an open source licence	Section 3.8
OS	Operating system	Section 6.2
PoC	Proof of Concept	Section 4.1
PUC	Product use case	Section 8.3
RFC	Request for Comments, a standard published by the IETF (Internet Engineering Task Force)	Section 18.2
SDK	Software development kit	Section 3.2
SRS	Software Requirements Specification	Section 4.1
URL	Uniform resource locator, a web address	Section 7
UTF-8	Unicode Transformation Format, 8 bit, the text encoding used for song maps	Section 18.2
V&V	Verification and validation	Section 12.1
W3C	World Wide Web Consortium	Section 18.2
WCAG	Web Content Accessibility Guidelines	Section 18.2

5 Relevant Facts And Assumptions

5.1 Relevant Facts

- Japanese is written without spaces between words and mixes kanji, hiragana, and katakana. Many kanji have more than one reading, and the correct reading depends on the context.
- In Anki, a note holds related information in fields, and its note type

defines those fields and the cards generated from it. Cards are organized into decks [2].

- AnkiConnect is an add-on for desktop Anki. By default it accepts requests only on `127.0.0.1`, port 8765, and it can be configured to require an API key [5].
- AnkiDroid provides an application programming interface (API) that other Android apps can use to add notes. It requires a permission that Android asks the user to grant at runtime [3]: com.ichi2.anki.permission.READ_WRITE_DATABASE.
- AnkiMobile, the iOS version of Anki, accepts new notes through an anki://x-callback-url/addnote URL from version 2.0.30 onwards [4]. Whether this can confirm that a note was created, as IN8 requires, has not been tested.
- Mobile operating systems control audio routing, system volume, and when background apps are suspended.

5.2 Business Rules

- **BR1 (Open Access):** no user account or payment is required to open a song map.
- **BR2 (Schema Adherence):** a song map must pass validation (Section 10) before the editor exports it or the learner app opens it.
- **BR3 (Copyright Compliance):** the project does not distribute audio or lyrics unless they are created by the team, in the public domain, or licensed for that use.

5.3 Assumptions

A1 Learners have an iOS or Android phone that meets the minimum operating system version of the Expo SDK release used.
Rationale: the app cannot run on older systems. This is a platform limit, not a design choice.

A2 Creators have the audio and lyric text for the songs they map, and the right to use them.
Rationale: the editor does not supply content (BR3, LG1).

A3 Learners who want to export vocabulary have Anki installed on the device that receives the notes.
Rationale: LingoLyrics is not a flashcard application. It hands vocabulary to Anki (G6).

A4 Software from other vendors (the operating systems, browsers, Anki, AnkiDroid, and AnkiConnect) behaves as described in its documentation.
Rationale: defects inside that software are outside the team's control. The app must still detect and report a failed handoff (IN8).

Song maps and their creators are not assumed to be free of errors or trustworthy. Requirements IN2, IN4, IN5, IM1, and IM2 handle incorrect or malicious song maps.

6 The Scope of the Work

6.1 The Current Situation

To study a song today, a learner plays it in a music app, reads the lyrics and a translation on a separate website, looks up words in a dictionary, and types useful words into Anki. Creators who want to share a song lesson have no common format, so each lesson is prepared by hand in documents or slides, and timing is not recorded.

6.2 The Context of the Work

Figure 1 shows the work context. Creators supply audio, lyrics, translations, and annotations to the editor, which produces a song map. Learners open song maps in the learner app, which plays the audio, shows the lyrics, and sends selected vocabulary to Anki.

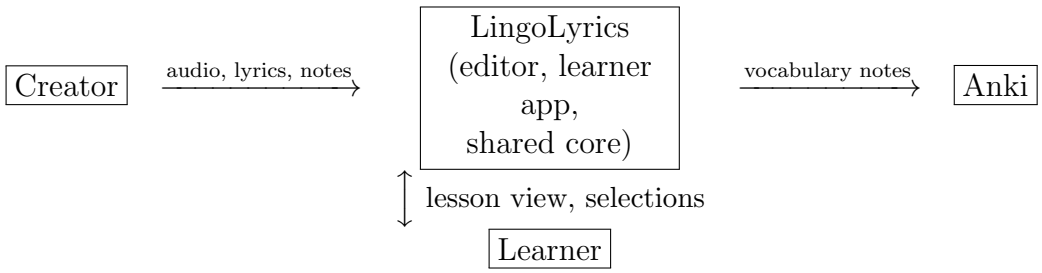


Figure 1: Context of the work. Each box is a person or system, and each arrow shows data flowing in the direction it points, labelled with what flows. The two way arrow means the learner both sends selections and receives the lesson view.

Table 4: Business events

Event	**Input**	**Output**
E1 Creator prepares a lesson	Audio file, lyric text, translations, annotations, licence information	Validated song map package
E2 Learner opens a lesson	Song map package	Lesson ready to play, or a message saying why it was rejected
E3 Learner plays a lesson	Play, pause, and seek actions	Audio with the current line and word highlighted
E4 Learner selects a word or line	Selection	Translation, reading, and annotation
E5 Learner sends a word to Anki	Selected word	Anki note, or a queued item and an error message

6.3 Work Partitioning

Table 4 lists the business events the work responds to.

The product is divided into three parts:

- **WP1 (Learner app):** lesson import, playback, synchronized highlighting, word selection, annotation display, and Anki export.
- **WP2 (Editor):** audio and lyric loading, timing, segmentation and transliteration review, translation and annotation entry, preview, and export.
- **WP3 (Shared core):** the song map schema, validation rules, and shared TypeScript types.

6.4 Specifying a Business Use Case (BUC)

- **BUC1 (Author a song map):** the creator loads audio and lyrics, sets line timings, reviews segmentation and readings, adds translations and annotations, and exports a validated song map (events E1).
- **BUC2 (Study a lesson):** the learner opens a song map, plays it while the current line is highlighted, selects words to read their meaning, and sends useful words to Anki (events E2 to E5).

7 Business Data Model and Data Dictionary

7.1 Business Data Model

A **song map** contains:

- **Metadata:** schema version, title, artist, target language, difficulty level, explicit content flag, audio file path, audio duration, measured audio loudness, audio and lyric source and licence, published flag, and export history.
- **Lyric lines:** start time, end time, original text, line translation, and an ordered list of words.
- **Words:** surface text, reading or transliteration, optional start and end time, translation, and a verified flag.

- **Annotations:** the word or line range they apply to, a contextual explanation, an optional note on register (how formal or offensive a term is), the name of the author, and the name of the reviewer.

7.2 Data Dictionary

Name	Content
`SongMap`	`schemaVersion` + `metadata` + `lines` + `annotations`. Must satisfy *valid* (Section 10).
`schemaVersion`	Version string *major.minor.patch*. A change in *major* means older apps cannot read the map (IN3).
`metadata`	`title`, `artist`, `language`, `difficulty`, `explicit`, `audioPath`, `audioDuration`, `loudness`, `source`, `published`, `exports`.
`loudness`	Integrated loudness of the audio in LUFS, measured by the editor (HS9).
`published`	Boolean. May be true only if the map satisfies *publishable* (Section 10, IN5).
`audioPath`	Path of the audio file relative to the song map package. Absolute paths and URLs are not allowed (IM2).
`source`	For both the audio and the lyrics: origin, licence type, and rights holder (LG1).
`exports`	List of (date, creator name, map version) entries, one per export (AU2).
`LyricLine`	`lineId`, `startTime`, `endTime`, `rawText`, `translatedText`, `words`.
`Word`	`text`, `reading`, optional `startTime` and `endTime`, `translation`, `verified`.
`Annotation`	`target` (line identifier and word range), `note`, optional `register`, `author`, `reviewedBy`. Names are entered by the creator and are not verified (no user accounts).
`VocabularyPayload`	`term`, `translation`, `contextSentence`, `mapId`, `lineId`. Sent to Anki (FR8).

Name	Content
Times	Seconds from the start of the audio, stored with resolution `TIME_RES`.

8 The Scope of the Product

8.1 Product Boundary

The product consists of the learner app, the editor, and the shared core. Anki, AnkiDroid, AnkiConnect, the phone and computer operating systems, and the source of the audio and lyric content are outside the boundary. The initial release has no user accounts, no server, and no cloud storage. Song map packages are moved between the editor and the learner app as files.

8.2 Product Use Case Table

PUC	Name	Actor	Outcome
PUC1	Play synchronized lesson	Learner	Lesson plays with the current line highlighted (FR1 to FR5).
PUC2	Inspect a word or line	Learner	Translation, reading, and annotation are shown (FR6, FR7).
PUC3	Send a term to Anki	Learner	A note is added to the LingoLyrics deck (FR8).
PUC4	Author a song map	Creator	A validated song map package is exported (FR9 to FR16).

8.3 Individual Product Use Cases (PUC's)

- **PUC1 (Play synchronized lesson):** the learner opens a song map package. The app validates it and shows the lesson. The learner presses play. The app plays the audio and highlights the current line and word. If validation fails, the app shows the reason and does not open the lesson.

- **PUC2 (Inspect a word or line):** the learner taps a word or line. The app shows the translation, reading, and annotation, if any, without stopping playback.
- **PUC3 (Send a term to Anki):** the learner taps *Add to Anki* on a selected word. The app builds a `VocabularyPayload`, sends it to Anki, and shows whether the note was created. If sending fails, the item is kept in a queue.
- **PUC4 (Author a song map):** the creator loads audio and lyric text, marks line start times while the audio plays, adjusts times, reviews the suggested segmentation and readings, enters translations and annotations, previews the lesson, and exports it. Export is refused until the map is valid.

9 Functional Requirements

9.1 Functional Requirements

Learner app

FR1 The learner app shall let the learner open a song map package from the device's storage and shall validate it (Section 10) before showing the lesson.
Rationale: Learners receive lessons as files in the initial release. Only valid maps can be played correctly.
Fit criterion: Every map in the acceptance suite opens. Every map in a set of invalid test maps is rejected with a message naming the failed rule.
Traces to: G3, PUC1, IN2, IN3, IN11.
Referenced by: RB1, SL1, UH6.

FR2 The learner app shall list the lessons it has opened, showing each lesson's title, artist, language, difficulty, and explicit content flag.
Rationale: Learners choose lessons by song and by level.
Fit criterion: All listed fields match the song map metadata for every acceptance suite map.
Traces to: G3, PUC1, HS6, HS7.
Referenced by: CP2.

FR3 The learner app shall provide play, pause, and seek controls, and a control that restarts the current line.
Rationale: Learners replay short sections to catch words.
Fit criterion: After each control is used, the reported audio position equals the expected position within `SYNC_TOL`.
Traces to: G1, PUC1, HS1.

FR4 During playback, the learner app shall highlight the line that is active at the current audio position, as defined by *active* in Section 10.
Rationale: This is the main function of the lesson player.
Fit criterion: In the acceptance suite, at least `SYNC_PASS` of line transitions occur within `SYNC_TOL` of their mapped start times.
Traces to: G1, PUC1, IN1, PA1.
Referenced by: PA1.

FR5 When the active line has word timings, the learner app shall also highlight the active word.
Rationale: Word timing helps learners match syllables to sounds, especially in fast songs.
Fit criterion: As FR4, applied to word start times.
Traces to: G1, PUC1.

FR6 The learner app shall display each line in its original script with the stored segmentation, and shall let the learner show or hide the transliteration and the line translation.
Rationale: Learners at different levels need different amounts of support.
Fit criterion: For every line in the acceptance suite, the displayed words and readings match the song map, and each layer can be turned on and off.
Traces to: G5, PUC2, IN4, IN12.
Referenced by: CU1, UH3.

FR7 When the learner selects a word or line, the learner app shall display its translation, reading, and contextual annotation, if present, without stopping playback.
Rationale: Meaning is clearer when shown next to the lyric (G2).
Fit criterion: Every annotated target in the acceptance suite shows its translation and annotation when selected.

Traces to: G2, PUC2, SL2.
Referenced by: SL2.

FR8 When the learner sends a selected word to Anki, the learner app shall create a note in the LingoLyrics deck whose *Term*, *Translation*, and *Context* fields contain the word, its translation, and the line it appears in.
Rationale: Connects song study to spaced repetition review (G6).
Fit criterion: For each acceptance suite language, a sent word appears in the test deck with all three fields correct.
Traces to: G6, PUC3, IN8, IN9, IN10, AC1.
Referenced by: OE2, SL3.

Editor

FR9 The editor shall let the creator load an audio file and lyric text, and open an existing song map for editing.
Rationale: Creators start from material they already have, and correct maps later.
Fit criterion: The test track's audio and lyrics load, and a previously exported map reopens with no data lost.
Traces to: G4, PUC4.
Referenced by: UH2.

FR10 The editor shall let the creator mark the start time of each line while the audio plays, and adjust start and end times by hand.
Rationale: Marking times while listening is faster than typing them (G4).
Fit criterion: Measured by G4 in usability testing.
Traces to: G1, G4, PUC4, IN7.
Referenced by: UH2.

FR11 For a supported language without spaces between words, the editor shall suggest a segmentation and readings for each line and let the creator accept or correct them.
Rationale: Automatic suggestions reduce work. Creator review corrects errors in lyric text that tokenizers handle poorly (PoC risk R2).
Fit criterion: At least `SEG_ACC` of suggested word boundaries in

SEG_SAMPLE test lines match a segmentation checked by the team.
Traces to: G4, G5, PUC4, IN4, PA3.
Referenced by: UH2.

FR12 The editor shall let the creator enter a translation for each line and word, contextual annotations for words and lines, and the lesson metadata listed in Section 7.
Rationale: These are the contents of a lesson.
Fit criterion: Each field in the data dictionary can be entered and appears in the exported map.
Traces to: G2, G4, PUC4, HS6, LG1.
Referenced by: UH2.

FR13 The editor shall provide a preview that plays the song map as the learner app will show it.
Rationale: Creators find timing and display errors before export.
Fit criterion: For the acceptance suite, the preview's line changes match the learner app's within SYNC_TOL.
Traces to: G1, G4, PUC4, IN7.
Referenced by: UH2.

FR14 The editor shall export a song map package only if the map is valid, and shall list every rule a map fails.
Rationale: Learners must receive only valid maps (G3, BR2).
Fit criterion: Export is refused for every invalid test map, and the listed rules match the rules the map breaks.
Traces to: G3, PUC4, IN2, IN4, IN5.
Referenced by: RB1, UH2, UH6.

FR15 The creator shall be able to complete FR9 to FR14 without editing the song map file directly.
Rationale: G4 requires that creators do not edit the file by hand.
Fit criterion: Measured by G4 in usability testing.
Traces to: G4, PUC4.
Referenced by: UH2.

Shared core

FR16 The editor and the learner app shall use the same validation code and schema version to decide whether a song map is valid.

Rationale: If the two disagree, the editor could export a map the app rejects (G3).
Fit criterion: The validator is a single shared module. Continuous integration (CI) validates every example map with it.
Traces to: G3, IN2, IN3.
Referenced by: MS1, MS2.

10 Formal Specification

This section defines a valid song map, the line that is active at a given audio position, and the learner app's playback modes. Constants are listed in Appendix A.

10.1 Song Map

A song map is a tuple $S = (v, d, L, N)$ where $v = (v_{maj}, v_{min}, v_{pat})$ is the schema version, $d \in \mathbb{R}^+$ is the audio duration in seconds, $L = \langle \ell_1, \ldots, \ell_n \rangle$ is the sequence of lines, and N is the set of annotations. Each line is $\ell_i = (s_i, e_i, W_i)$ with start time s_i, end time e_i, and words $W_i = \langle w_{i,1}, \ldots, w_{i,m_i} \rangle$. A word $w_{i,j}$ has a verified flag $ver_{i,j}$ and optional times $s_{i,j}, e_{i,j}$. An annotation $a \in N$ has a translation $a.tr$, a note $a.note$, an author $a.auth$, and a reviewer $a.rev$.

S is valid, written $valid(S)$, when all of conditions V1 to V8 hold:

V1 (version) $v_{maj} \in$ `SUPPORTED_MAJOR` (IN3).

V2 (at least one line) $n \geq 1$.

V3 (lines inside the audio) $\forall i \in [1..n] : 0 \leq s_i < e_i \leq d$.

V4 (lines ordered, no overlap) $\forall i \in [1..n-1] : e_i \leq s_{i+1}$.

V5 (words inside their line) for every word $w_{i,j}$ that has times, $s_i \leq s_{i,j} < e_{i,j} \leq e_i$.

V6 (segmentation verified) $\forall i, j : ver_{i,j}$ (IN4).

V7 (annotations complete) $\forall a \in N : a.tr \neq \varepsilon \wedge a.note \neq \varepsilon$.

V8 (audio reference and limits) the audio path is relative and resolves inside the package (IM2), $|d - d_{audio}| \leq$ `AUDIO_DUR_TOL` (IN11), the map size is at most `MAX_MAP_SIZE`, and its nesting depth is at most `MAX_NESTING` (IM1).

Here d_{audio} is the duration of the decoded audio file and ε is the empty string. The JSON Schema [10] checks structure and types. The shared validator checks the remaining conditions (FR16).

A valid map may be marked as published only when every annotation has a reviewer other than its author (IN5):

$$\mathit{publishable}(S) \equiv \mathit{valid}(S) \wedge \forall a \in N : a.\mathit{rev} \neq \varepsilon \wedge a.\mathit{rev} \neq a.\mathit{auth}.$$

10.2 Active Line and Synchronization

For an audio position p, the active line is

$$\mathit{active}(p) = \begin{cases} i & \text{if } s_i \leq p < e_i \\ \bot & \text{if no line satisfies } s_i \leq p < e_i \end{cases}$$

which is well defined because valid lines do not overlap. Let h_i be the audio position at which the learner app changes the highlight to line i. A transition is on time when $|h_i - s_i| \leq$ `SYNC_TOL`. FR4 requires that, over all transitions in the acceptance suite,

$$\frac{|\{i : |h_i - s_i| \leq \texttt{SYNC_TOL}\}|}{\text{number of transitions}} \geq \texttt{SYNC_PASS}.$$

10.3 Playback Modes

The learner app is in one of the modes *NoLesson*, *Ready*, *Playing*, and *Paused*. The initial mode is *NoLesson*. Table 6 gives the transitions. Events not listed for a mode leave the mode unchanged. In *Playing* and *Paused*, the highlighted line is $\mathit{active}(p)$, where p is the position reported by the audio player.

No transition enters *Playing* except a learner action, which is HS1, and every such transition fades the audio in, which is HS8. The transition on audio output disconnection is HS2. Locking the screen or moving the app to the background is not an event, so it leaves *Playing* unchanged (HS3).

Table 6: Learner app playback mode transitions

Mode	Event	Action	New mode
NoLesson	open S, $valid(S)$	$p := 0$	Ready
NoLesson	open S, $\neg valid(S)$	show failed rules	NoLesson
Ready	learner presses play	start audio, fade in over `FADE_IN`	Playing
Playing	learner presses pause	stop audio	Paused
Playing	audio output disconnected	stop audio within `DISCONNECT_PAUSE`	Paused
Playing	end of audio	$p := d$	Paused
Playing, Paused	seek to t	$p := t$. Highlight $active(t)$	unchanged
Paused	learner presses play	start audio, fade in over `FADE_IN`	Playing
any	close lesson	stop audio	NoLesson

11 Look and Feel Requirements

11.1 Appearance Requirements

LF1 The active line and word shall be distinguishable from the other lines by a cue other than colour alone.
Rationale: Learners with colour vision deficiency must be able to follow the lesson (Hazard Analysis H2.5).
Fit criterion: The active line can be identified in a greyscale screenshot and in a colour blindness simulation.
Traces to: G1, HS5.

LF2 In the lesson view, the lyric area shall take up more of the screen than the playback controls and menus.
Rationale: The lyrics are the content the learner studies.
Fit criterion: Measured on screenshots of the lesson view on a phone screen.
Traces to: G1, G2.

11.2 Style Requirements

LF3 The learner app shall follow the platform's interface conventions for controls and navigation on iOS and Android.
Rationale: Learners already know these conventions, which reduces learning time (UH3).
Fit criterion: Usability test participants operate the controls without instruction.
Traces to: UH3.

12 Usability and Humanity Requirements

12.1 Ease of Use Requirements

UH1 A learner shall be able to open a lesson that has already been imported and start playback in at most `MAX_TAPS_PLAY` taps from the app's home screen.
Rationale: Learners use the app in short sessions.
Fit criterion: Counted on the release build.
Traces to: G1, PUC1.

UH2 At least `CREATE_PASS` test creators shall produce a complete, valid map for the test track within `CREATE_TIME`, without editing the map file by hand.
Rationale: Goal G4.
Fit criterion: Measured in the usability study described in the V&V Plan.
Traces to: G4, PUC4, FR9 to FR15.
Referenced by: MS3.

12.2 Personalization and Internationalization Requirements

UH3 The learner shall be able to show or hide the transliteration and translation layers, and the app shall remember the choice between sessions.
Rationale: Learners reduce support as they improve.
Fit criterion: Settings persist after the app is closed and reopened.

Traces to: G5, FR6.
Referenced by: LF3.

UH4 The user interface of the learner app and the editor shall be in English in the initial release. Lesson content shall support at least `LANG_COUNT` target languages.
Rationale: The team and the test users work in English. G5 requires `LANG_COUNT` languages.
Fit criterion: All interface text is English. The acceptance suite contains `LANG_COUNT` languages.
Traces to: G5.

12.3 Learning Requirements

UH5 A learner using the app for the first time shall be able to open a lesson, play it, display a word's annotation, and send the word to Anki within `LEARN_TIME` without help.
Rationale: Learners use the app on their own, without training.
Fit criterion: At least `CREATE_PASS` of usability test participants succeed.
Traces to: G1, G2, G6.
Referenced by: MS3.

12.4 Understandability and Politeness Requirements

UH6 Every error message shall state what failed and what the user can do next, in plain language and without blaming the user.
Rationale: Learners and creators are not expected to know the song map format or Anki internals.
Fit criterion: Review of every error message string against this rule.
Traces to: FR1, FR14, IN3, IN8.

12.5 Accessibility Requirements

UH7 Every control in the learner app shall have a label read by the platform screen reader, and lyric text shall scale with the device's text size setting.
Rationale: Learners with low vision must be able to use the app.

Fit criterion: Each control is announced by VoiceOver and TalkBack. Lyrics remain readable at the largest system text size.
Traces to: HS5.

The accessibility requirements HS4 (flashing) and HS5 (colour and text size) are listed under Safety-Critical Requirements.

13 Performance Requirements

13.1 Speed and Latency Requirements

SL1 A lesson shall be ready to play within `LOAD_TIME` of the learner opening it.
Rationale: Taken from the PoC pass criterion for risk R2.
Fit criterion: Measured on the test phone for each acceptance suite lesson.
Traces to: G1, G5, FR1.

SL2 The translation and annotation of a selected word or line shall be displayed within `SELECT_RESPONSE` of the selection.
Rationale: The learner reads the meaning while the line is still playing.
Fit criterion: Measured on the test phone with instrumented timestamps.
Traces to: G2, FR7.
Referenced by: FR7.

SL3 The learner app shall report the result of sending a word to Anki within `ANKI_TIMEOUT`. If Anki has not confirmed by then, the item shall be treated as failed and queued (IN8).
Rationale: The learner should not wait without feedback.
Fit criterion: Tested with Anki available and unavailable.
Traces to: G6, FR8, IN8.

13.2 Safety-Critical Requirements

These requirements come from the Hazard Analysis. The FMEA row each one mitigates is given in brackets.

HS1 The learner app shall not start audio playback without an explicit action by the learner.
Rationale: Unexpected audio can cause hearing discomfort with headphones.
Fit criterion: Opening, resuming, and returning to a lesson never starts audio (Table 6).
Traces to: Hazard H1.2.
Referenced by: FR3.

HS2 The learner app shall pause playback within `DISCONNECT_PAUSE` when the active audio output device is disconnected.
Rationale: Prevents audio from playing aloud through the phone speaker.
Fit criterion: Tested by unplugging wired and disconnecting Bluetooth headphones during playback on iOS and Android.
Traces to: Hazard H1.2.

HS3 Once a lesson has been started, the learner app shall continue playing it to the end, with no interaction required, while the screen is locked or the app is in the background. The app shall show a notice on first launch advising against using the screen while driving, cycling, or crossing traffic.
Rationale: Learners use the app while moving. A lesson that can be followed by listening alone lets them put the phone away. A separate listening mode designed for use while moving is future work (Hazard Analysis, Roadmap).
Fit criterion: On iOS and Android, a full acceptance suite lesson plays to the end with the screen locked after the learner presses play. The notice appears on the first launch after installation and not on later launches.
Traces to: Hazard H1.4.
Referenced by: OE1.

HS4 Lyric highlighting shall not produce more than `MAX_FLASHES` large area flashes in any period of one second, and shall follow the operating system's reduce motion setting.
Rationale: Rapid flashing can trigger seizures in photosensitive people (Web Content Accessibility Guidelines (WCAG) 2.1, Success Criterion 2.3.1 [6, 8]).

Fit criterion: Count of highlight changes per second in the fastest acceptance suite song, and a check with reduce motion turned on.
Traces to: Hazard H2.4.
Referenced by: ST1.

HS5 The learner app shall indicate the active line and word with a cue other than colour alone, and shall respect the device's text size setting.
Rationale: See LF1 and UH7.
Fit criterion: As LF1 and UH7.
Traces to: Hazard H2.5.
Referenced by: LF1, ST1, UH7.

HS6 Each song map shall include an explicit content flag. Lessons flagged as explicit shall show a warning before opening and can be hidden through a learner setting. Annotations of vulgar or offensive terms shall include a note describing their register.
Rationale: Learners may otherwise use an offensive term without knowing it.
Fit criterion: Schema requires the flag. A flagged test lesson shows the warning and is hidden when the setting is on.
Traces to: Hazard H3.2.
Referenced by: FR2, FR12.

HS7 The learner app shall not use streak loss penalties, countdown timers, or unsolicited push notifications to encourage use, and shall show each lesson's difficulty level before it is opened.
Rationale: Avoids stress and compulsive use.
Fit criterion: Design review of the release build.
Traces to: Hazards H1.4, H3.3.
Referenced by: FR2.

HS8 When playback starts or resumes, the learner app shall raise the playback volume from silence to its set level over `FADE_IN`.
Rationale: A sudden start at full level is uncomfortable with headphones, and a short fade gives the learner time to lower the volume.
Fit criterion: A recording of the app's audio output at the start of playback and after each resume rises from silence to the set level over

`FADE_IN`, within 0.1 s, on iOS and Android.
Traces to: Hazard H1.2.

HS9 The editor shall measure the integrated loudness of each track's audio, as defined in ITU-R BS.1770 [12], and display it to the creator. Before export, it shall warn the creator when the loudness differs from `TARGET_LOUDNESS` by more than `LOUDNESS_TOL`. The measured value shall be stored in the song map's `loudness` field.
Rationale: Tracks mastered at very different levels make one lesson much louder than the previous one at the same volume setting.
Fit criterion: For test tracks with known integrated loudness, the displayed value is within 1 LU of the known value. A test track outside the tolerance produces the warning before export.
Traces to: Hazard H1.2.

13.3 Precision or Accuracy Requirements

PA1 Song map times shall be stored with a resolution of `TIME_RES` or finer.
Rationale: Storage resolution must be much finer than `SYNC_TOL` so that rounding does not use up the timing tolerance.
Fit criterion: Schema check on the time fields.
Traces to: G1, FR4.
Referenced by: FR4.

PA2 Line highlighting accuracy shall meet the fit criterion of FR4.
Rationale: Goal G1.
Fit criterion: As FR4.
Traces to: G1.

PA3 Automatically suggested segmentation shall meet the fit criterion of FR11.
Rationale: Taken from the PoC pass criterion for risk R2.
Fit criterion: As FR11.
Traces to: G5.
Referenced by: FR11.

13.4 Robustness or Fault-Tolerance Requirements

RB1 The learner app and the editor shall not crash when given an invalid or malformed song map. They shall reject it with a message (UH6).
Rationale: Song maps are files that can be damaged or altered.
Fit criterion: A set of malformed test maps (truncated, wrong types, oversized, deeply nested) is rejected without a crash.
Traces to: FR1, FR14, IM1.

RB2 The learner app shall play lessons that are already on the device without a network connection.
Rationale: The product has no server, and learners use the app in transit.
Fit criterion: All acceptance suite lessons play in airplane mode.
Traces to: G1.

13.5 Capacity Requirements

CP1 The learner app and the editor shall support songs up to `MAX_AUDIO_LENGTH` long with up to `MAX_LESSON_LINES` lines.
Rationale: Covers typical popular songs.
Fit criterion: A test map at both limits opens, plays, and meets FR4.
Traces to: G1, G4.

CP2 The learner app shall keep at least `LIBRARY_SIZE` lessons.
Rationale: Learners keep several songs in progress.
Fit criterion: `LIBRARY_SIZE` lessons imported and listed on the test phone.
Traces to: FR2.

13.6 Scalability or Extensibility Requirements

SC1 Adding a further target language shall not require a change to the major version of the song map schema.
Rationale: The stretch goal of a third language. Existing maps must remain valid.
Fit criterion: The schema's fields that are specific to a language are optional and keyed by language code. Reviewed when the design is

complete.
Traces to: G5, LO1.

13.7 Longevity Requirements

LO1 A learner app version shall open every valid song map whose major schema version it supports, regardless of minor or patch version.
Rationale: Lessons made earlier must keep working.
Fit criterion: CI opens maps of every released minor version with the current app.
Traces to: G3, IN3.
Referenced by: OE5, SC1.

14 Operational and Environmental Requirements

14.1 Expected Physical Environment

OE1 The learner app shall be usable on a phone held in one hand, with headphones, in indoor and outdoor lighting.
Rationale: Learners study in short sessions in varied places.
Fit criterion: All lesson controls reachable with one hand. Checked during usability testing.
Traces to: HS3.

14.2 Wider Environment Requirements

The product has no wider environment requirements. It does not run on shared infrastructure.

14.3 Requirements for Interfacing with Adjacent Systems

OE2 The learner app shall send vocabulary to Anki using at least one of: the AnkiDroid API on Android, AnkiConnect with desktop Anki, or an export file that the learner imports into Anki.
Rationale: No single method works on every platform. The PoC (risk R3) selects the method. The export file is the fallback in the

Development Plan.
Fit criterion: FR8's fit criterion is met with the selected method.
Traces to: G6, FR8, AC1, AC2, PR2.

OE3 The editor shall read audio in at least one format that the learner app can play on both iOS and Android, and shall export only such formats.
Rationale: A map whose audio cannot play on one platform fails G3.
Fit criterion: Each acceptance suite package plays on an iOS and an Android test device.
Traces to: G3, IN11.

14.4 Productization Requirements

OE4 The learner app shall be installable on iOS and Android test devices as an Expo development or preview build, and the editor shall run from a URL without installation.
Rationale: Store publication is not required for the capstone.
Fit criterion: Installed and run on one iOS and one Android device. Editor opened in each supported browser.
Traces to: MC2.
Referenced by: MS4.

14.5 Release Requirements

OE5 Each release shall be tagged on the `main` branch and shall state the song map schema version it supports.
Rationale: Creators and learners need to know which maps a release can open (LO1).
Fit criterion: Check of release tags and notes.
Traces to: LO1.

15 Maintainability and Support Requirements

15.1 Maintenance Requirements

MS1 Every example song map in the repository shall be validated in CI on each pull request.
Rationale: Detects schema changes that break existing maps.

Fit criterion: CI configuration and run history.
Traces to: G3, FR16.

MS2 The shared core shall have statement coverage by unit tests of at least `COVERAGE`.
Rationale: The validator decides what learners can open. Errors there affect every lesson.
Fit criterion: Coverage report from CI.
Traces to: FR16, IN2.

15.2 Supportability Requirements

MS3 The project shall provide written instructions for learners (importing lessons, setting up Anki) and for creators (using the editor, licence fields).
Rationale: There is no support staff.
Fit criterion: Usability test participants complete their tasks using only the instructions (UH2, UH5).
Traces to: UH2, UH5, AC2.

15.3 Adaptability Requirements

MS4 The learner app shall run on both iOS and Android from one code base.
Rationale: MC2. Learners use both platforms.
Fit criterion: The same release build runs on one device of each platform.
Traces to: MC2, OE4.

16 Security Requirements

These requirements come from the Hazard Analysis. The FMEA row each one mitigates is given in brackets.

16.1 Access Requirements

AC1 The learner app shall only add notes, and only to the LingoLyrics deck. It shall not edit or delete existing notes, decks, or note types in

the learner's Anki collection.
Rationale: The learner's existing study data must not be damaged.
Fit criterion: Code review of every Anki call. An integration test confirms the test collection is unchanged apart from added notes.
Traces to: Hazard H6.3.
Referenced by: FR8, OE2.

AC2 If AnkiConnect is used, the product and its setup instructions shall allow only the specific origin the product requires, never a wildcard origin, and shall require AnkiConnect's API key (`apiKey` [5]). The setup instructions shall warn the learner before any step that makes AnkiConnect listen on an address other than `127.0.0.1`.
Rationale: A wildcard origin lets any web page change the learner's collection. AnkiConnect listens only on `127.0.0.1` by default, so a phone can reach it only if it is opened to the network, where any device could send it requests.
Fit criterion: Review of the setup instructions and default configuration. A request without the API key is refused.
Traces to: Hazard H6.5.
Referenced by: MS3, OE2.

16.2 Integrity Requirements

IN1 The learner app shall drive lyric highlighting from the audio player's reported position and shall realign the highlight after every seek, pause, resume, and return from the background.
Rationale: A separate timer drifts from the audio.
Fit criterion: FR4's fit criterion holds after seeks and after returning from the background.
Traces to: Hazard H1.1.
Referenced by: FR4.

IN2 The song map validator shall check, in addition to the schema, the conditions of *valid* in Section 10. The editor shall run it before export and the learner app shall run it on import.
Rationale: Schema validation alone accepts maps with overlapping times or times outside the audio.
Fit criterion: One invalid test map per condition is rejected by both

the editor and the app.
Traces to: Hazards H1.1, H4.1.
Referenced by: FR1, FR14, FR16, MS2.

IN3 Each song map shall declare its format version. The learner app shall refuse to open a map with an unsupported major version and shall tell the learner why.
Rationale: Prevents fields from being ignored or misread.
Fit criterion: A map with an unsupported major version is rejected with a message.
Traces to: Hazard H4.2.
Referenced by: FR1, FR16, LO1, UH6.

IN4 Song maps shall store word segmentation and transliteration explicitly, and the learner app shall display the stored values without segmenting again at runtime. The editor shall mark automatically generated values as unverified until the creator confirms them, and shall not export a map containing unverified lines.
Rationale: Tokenizers make errors on lyrics. Creator review catches them once.
Fit criterion: The app contains no tokenizer. Export of a map with an unverified line is refused.
Traces to: Hazards H2.1, H2.2.
Referenced by: FR6, FR11, FR14.

IN5 Each annotation shall record the name of its author and the name of its reviewer. The editor shall not mark a map as published while any annotation has no reviewer or has its author as its reviewer (*publishable*, Section 10).
Rationale: Incorrect annotations teach the wrong meaning. Because the initial release has no user accounts, names cannot be verified. The residual risk is accepted in the Hazard Analysis.
Fit criterion: A map with an annotation that has no reviewer, or whose reviewer is its author, cannot be marked published.
Traces to: Hazard H3.1.
Referenced by: CU2, FR14.

IN6 The editor shall autosave the current draft at least every `AUTOSAVE_INT`, offer to restore it when reopened, warn before closing with unsaved

changes, provide undo and redo for all edits, and ask for confirmation before any operation that changes more than one line.
Rationale: Timing work takes time and is easy to lose.
Fit criterion: Closing the tab during an edit and reopening restores the draft. Each bulk operation asks for confirmation.
Traces to: Hazards H5.1, H5.2.

IN7 The editor shall let the creator set an audio latency offset, either by entering it or by tapping along to a test click track, and shall apply it to recorded timestamps. It shall also provide a preview that plays the map as the learner app will (FR13).
Rationale: Browser and Bluetooth output latency shifts every recorded time.
Fit criterion: With calibration applied, preview line changes meet FR4's fit criterion.
Traces to: Hazard H5.3.
Referenced by: FR10, FR13.

IN8 The learner app shall report a vocabulary item as sent only after Anki confirms that the note was created. Items that have not been confirmed shall be stored on the device, kept across app restarts, and available to retry or export to a file.
Rationale: Learners must not lose words they believe were saved.
Fit criterion: With Anki unavailable, sent items remain queued after the app restarts and are delivered on retry.
Traces to: Hazards H6.4, H7.1.
Referenced by: FR8, SL3, UH6.

IN9 Before adding a note, the learner app shall check the LingoLyrics deck for an existing note with the same term and context, and shall disable the send control while a request is in progress.
Rationale: Prevents duplicate cards.
Fit criterion: Sending the same word twice creates one note.
Traces to: Hazard H6.1.
Referenced by: FR8.

IN10 The learner app shall fill Anki note fields by field name, shall stop with a message if the note type lacks the *Term*, *Translation*, or *Context* field, and shall send all text as UTF-8.

Rationale: Prevents misplaced or garbled fields.
Fit criterion: Japanese test words appear correctly in the right fields. A note type missing a field produces the message.
Traces to: Hazard H6.2.
Referenced by: FR8.

IN11 On import, the learner app shall check that the audio file referenced by a map exists, can be decoded, and has a duration within `AUDIO_DUR_TOL` of the map's `audioDuration`, and shall reject the lesson with a message if not.
Rationale: Finds broken packages before playback.
Fit criterion: Packages with a missing, corrupt, or mismatched audio file are rejected.
Traces to: Hazard H1.3.
Referenced by: FR1, OE3.

IN12 The learner app shall include fonts covering every script used by its supported languages, and all song maps shall be read and written as UTF-8.
Rationale: Device fonts may not cover every script.
Fit criterion: Every acceptance suite line renders with no boxes for missing glyphs on both platforms.
Traces to: Hazard H2.3.
Referenced by: FR6.

16.3 Privacy Requirements

PR1 The initial release shall not send any data off the device, other than notes the learner explicitly sends to Anki and error reports the learner chooses to share (AU1). Any analytics or crash reporting added later shall stay off unless the learner turns it on.
Rationale: Vocabulary history and listening habits are personal.
Fit criterion: Dependency review and network capture during a test session show no other outbound traffic.
Traces to: Hazard H7.2.

PR2 The learner app shall request the AnkiDroid permission only when the learner first sends a word, with an explanation of why it is needed, and shall request no other sensitive permissions.

Rationale: Learners should understand what they grant.
Fit criterion: Review of the app manifest and the permission prompt sequence.
Traces to: Hazard H6.6.
Referenced by: OE2.

16.4 Audit Requirements

AU1 The learner app shall let a learner prepare a report of a suspected error on any lyric line. The report shall contain the map identifier, map version, line, and the learner's comment. The app shall pass the report to the operating system's share sheet, so that the learner decides whether and where to send it, and shall not send it itself.
Rationale: Creators need to know about incorrect annotations. The product has no backend, so the learner chooses the channel (for example, email or the project's issue tracker).
Fit criterion: A report prepared from a test lesson contains all four items and opens in the share sheet on iOS and Android. Network capture shows that the app sends nothing when the learner cancels the share sheet.
Traces to: Hazard H3.1.

AU2 Each song map shall record a version number and, for every export, the date and the creator who exported it.
Rationale: Allows changes to a lesson to be traced.
Fit criterion: Exporting a map twice adds two entries to `exports`.
Traces to: Hazards H3.1, H5.2.

16.5 Immunity Requirements

IM1 The song map parser in both the editor and the learner app shall reject a map whose size exceeds `MAX_MAP_SIZE` or whose nesting depth exceeds `MAX_NESTING` before parsing it fully, and both shall display all map text as plain text, never as HTML.
Rationale: Prevents denial of service and script injection through crafted maps.
Fit criterion: Oversized, deeply nested, and test maps containing script are rejected or shown as plain text.

Traces to: Hazard H4.3.
Referenced by: RB1.

IM2 Song maps shall refer to audio and other resources only by relative paths that resolve inside the map package. The validator shall reject absolute paths, remote URLs, and any path that leaves the package, such as one containing a `..` segment. The learner app shall extract each imported package only into a new folder that it owns.
Rationale: Prevents the app from contacting unknown servers and stops a crafted package from reading or writing files outside its own folder.
Fit criterion: Maps with absolute paths, URLs, or paths that leave the package are rejected, and test packages with such paths change no file outside a new folder owned by the app.
Traces to: Hazards H4.4, H4.5, H7.2.

IM3 The project shall pin the exact version of every library from other vendors in a committed lock file. CI shall run an automated vulnerability check on every pull request and fail when a high or critical vulnerability is reported. Every new dependency and every major version change shall be reviewed by a team member other than the one who added it before it is merged.
Rationale: The app and the editor depend on many npm packages, and a compromised or vulnerable package runs with full access to their data.
Fit criterion: Both projects have a committed lock file, the CI log shows the vulnerability check on every pull request, and every pull request that changes dependencies has an approval from a second team member.
Traces to: Hazard H7.3.

17 Cultural Requirements

17.1 Cultural Requirements

CU1 Japanese romanization shall follow modified Hepburn romanization.
Rationale: A single standard makes readings consistent across lessons. Hepburn is the system most used in learning material written in

English.
Fit criterion: Romanization of the Japanese acceptance suite map matches Hepburn for every word.
Traces to: G5, FR6.

CU2 Contextual annotations for culturally specific references (for example, place names, holidays, or idioms) shall explain the reference rather than only translate it.
Rationale: A literal translation can hide the meaning.
Fit criterion: Review of the acceptance suite annotations.
Traces to: G2, IN5.

The explicit content requirement HS6 also addresses cultural sensitivity.

18 Compliance Requirements

18.1 Legal Requirements

LG1 The editor shall require source and licence information for the audio and lyrics before a map can be exported, and the learner app shall show these credits for each lesson.
Rationale: Distributing unlicensed lyrics or recordings creates legal liability (BR3).
Fit criterion: Export is refused when licence fields are empty. Credits appear in each acceptance suite lesson.
Traces to: Hazard H5.4.
Referenced by: FR12.

Intended Data Flow and Privacy Obligations

The initial release is designed so that the team does not receive any personal data. The intended data flow is:

- The learner's lessons, display preferences, and queue of vocabulary waiting for Anki are stored only on the learner's device (IN8, UH3).
- Vocabulary leaves the app only when the learner sends it to Anki. The note goes to Anki on the same device, or to desktop Anki through AnkiConnect, and is then stored in the learner's own Anki collection.

What happens to the collection after that, including synchronization through AnkiWeb, is controlled by the learner and Anki, not by LingoLyrics.

- Error reports leave the device only through the share sheet, to a destination the learner chooses (AU1).
- Song maps contain the names of the creators and reviewers (AU2, IN5), and these names are distributed with each map.
- The editor stores drafts in the creator's browser storage (IN6).

This data flow limits the product's privacy obligations, but does not remove them: the app stores vocabulary history on the device, it writes into the learner's Anki collection, and song maps carry creators' names. The team has not yet determined which privacy laws and platform policies apply. Before the app is distributed outside the team and test participants, the team shall confirm this, including Canadian privacy law and the privacy disclosures required by the Apple App Store and Google Play, and shall check it again whenever the data flow changes (see Open Issues).

18.2 Standards Compliance Requirements

ST1 The learner app's text and highlight colours shall meet WCAG 2.1 level AA contrast (Success Criterion 1.4.3 [7]), and highlighting shall meet Success Criterion 2.3.1 [8] (HS4).
Rationale: WCAG [6] is the W3C's accessibility standard for web content, and the W3C publishes guidance on applying it to software that is not web content, such as mobile apps (WCAG2ICT [9]).
Fit criterion: Contrast checker on every text and highlight colour pair.
Traces to: HS4, HS5.

ST2 The song map schema shall be written in JSON Schema Draft 2020-12 [10], and song maps shall be valid JSON (RFC 8259 [11]) encoded in UTF-8.
Rationale: Standard formats allow existing validators and editors to be used.
Fit criterion: The schema validates against the Draft 2020-12 metaschema (https://json-schema.org/draft/2020-12/schema).
Traces to: G3, MC3.

19 Open Issues

- **Anki handoff method.** Whether the learner app uses the AnkiDroid API, AnkiConnect, an export file, or a combination depends on the PoC (risk R3). On iOS, AnkiMobile's `addnote` URL scheme [4] is a candidate, but whether it can confirm that a note was created (IN8) has not been tested. Affects OE2, AC2, PR2, IN8 to IN10.
- **Mobile timing accuracy.** Whether Expo Audio reports position accurately enough for FR4 depends on the PoC (risk R1).
- **Second language.** Japanese is the language whose writing system is not Latin. The second language for G5 has not been chosen.
- **Japanese tokenizer.** kuromoji.js is the current choice for FR11. Sudachi is more accurate but needs a separate service. Because segmentation runs in the editor (IN4), either is possible.
- **Lesson distribution.** The initial release moves song map packages as files. Whether lessons should also be bundled with the app for the demonstration is undecided.
- **Loudness target.** HS9 measures loudness with ITU-R BS.1770. The values of `TARGET_LOUDNESS` and `LOUDNESS_TOL` are initial and will be confirmed with the acceptance suite tracks. Whether the editor should also normalize the audio, rather than only warn, is undecided.
- **Privacy obligations.** The privacy laws and app store privacy disclosures that apply to the intended data flow (Compliance Requirements) have not been confirmed. This must be done before the app is distributed outside the team and test participants.

20 Off-the-Shelf Solutions

20.1 Ready Made Products

- **LyricsTraining** [13]: a website and mobile app in which learners fill in missing words in the lyrics of music videos. Japanese songs can be shown in rōmaji. We found no contextual annotations or Anki export.

- **Musixmatch** [14]: a lyrics service that shows lyrics synchronized with the music, line by line and for some songs word by word, with translations contributed by its users for some songs. It is designed for listening rather than study, and we found no vocabulary export.
- **Language Reactor** [15]: a browser extension that shows dual subtitles on video sites such as Netflix and YouTube, lets learners look up and save words, and in its paid version exports saved words for use in Anki. It depends on the video's existing subtitles and does not provide annotations written by creators.

None of these combines annotations written by creators, verified segmentation for scripts other than Latin, and Anki export, which are goals G2, G5, and G6. These descriptions are based on each product's public website and store listings, checked on October 18, 2026. Features may change.

20.2 Reusable Components

- kuromoji.js for Japanese segmentation and readings (FR11).
- A JSON Schema validator library for FR16.
- Expo Audio and Howler.js for playback (FR3, FR13).
- Anki, AnkiDroid, and AnkiConnect as the flashcard system (FR8).

20.3 Products That Can Be Copied

The LRC lyric file format stores lines as `[mm:ss.xx] text`, and its extended form adds word timestamps. The song map's timing fields can follow the same model, and the editor may import LRC files in a later release (stretch goal: song map import and export).

21 New Problems

21.1 Effects on the Current Environment

Learners will move some song study from several separate tools into one app. Creators will need to record licence information they may not have recorded

before (LG1).

21.2 Effects on the Installed Systems

The learner app adds notes to the learner's Anki collection and may create a LingoLyrics deck and note type (AC1). If AnkiConnect is used, the learner must change its configuration (AC2).

21.3 Potential User Problems

- Learners may import songs they do not have the right to use. The product cannot prevent this for personal files (BR3 applies only to content the project distributes).
- Learners may rely on transliteration and not learn to read the original script. UH3 lets them hide it.
- Learners may use the app while moving (HS3).

21.4 Limitations in the Anticipated Implementation Environment That May Inhibit the New Product

- The interval at which Expo Audio reports playback position limits highlight accuracy (risk R1).
- iOS does not provide a confirmed way for another app to add notes to Anki directly (OE2).
- Mobile operating systems may suspend apps in the background, which affects highlight alignment after return (IN1).

21.5 Follow-Up Problems

Changes to the song map schema after lessons are published will require older maps to remain readable (LO1) or a conversion tool (stretch goal).

22 Tasks

22.1 Project Planning

Development tasks are divided by component ownership, as in the Development Plan:

- Learner app playback and lyric sync: Omar Bakr
- Song map editor and schema: Raymond Ma
- Language processing (segmentation, romanization): Rhea Gokhale
- Anki integration and CI/CD: Aansh Kotian

22.2 Planning of the Development Phases

Work is tracked on a GitHub Projects board. The phases are:

- **Phase 1 (Fall 2026, to the PoC demonstration, November 23 to December 4):** audio synchronization (R1), Japanese segmentation (R2), and Anki handoff (R3) on one test phone.
- **Phase 2 (January to February 2027, to Revision 0):** complete learner app and editor, the requirements in this document marked for the capstone in the Hazard Analysis roadmap, and usability testing.
- **Phase 3 (February to March 2027, to Revision 1 and the final documentation on March 31):** fixes from usability testing, remaining requirements, and the EXPO demonstration.

22.3 Requirement Priorities and Phase In Plan

Each requirement has one of three priorities. **Priority 1** requirements address a PoC risk (R1 to R3) and must work before anything else can be built on them. **Priority 2** requirements are needed to meet a goal (G1 to G6) or to mitigate a hazard. **Priority 3** requirements improve usability, are measured in usability testing, or concern release and maintenance. Table 7 gives the date by which each requirement will be implemented and tested. Stretch goals in the Waiting Room are not scheduled.

Table 7: Requirement priorities and phase in dates

Priority	Done by	Requirements
1	November 23, 2026 (PoC demonstration)	MC1 to MC3, FR3, FR4, FR6, FR8 (one handoff method), FR11, IN1, IN4, SL1, PA1 to PA3, CU1, OE2, ST2
2	February 1, 2027 (Revision 0 demonstration)	FR1, FR2, FR5, FR7, FR9, FR10, FR12 to FR16, HS1 to HS9, AC1, AC2 (if AnkiConnect is used), IN2, IN3, IN5 to IN12, PR1, PR2, AU1, AU2, IM1 to IM3, LG1, SL2, SL3, RB1, RB2, CP1, LO1, OE3, OE4, MS1, MS2, MS4, UH4, UH6
3	March 15, 2027 (Revision 1 demonstration)	LF1 to LF3, UH1 to UH3, UH5, UH7, CP2, SC1, OE1, OE5, MS3, CU2, ST1

If a PoC risk is not resolved by November 23, the fallback in the Development Plan applies, and the Priority 1 requirements it affects are revised before work on Priority 2 begins. Requirements measured by usability testing (UH1, UH2, UH5) are implemented in Phase 2 but are confirmed in Phase 3, after the usability study.

23 Migration to the New Product

23.1 Requirements for Migration to the New Product

LingoLyrics is a new product and does not replace a deployed application, so no user accounts, lesson libraries, or other data need to be migrated for the initial release.

Creators produce new song maps with the editor, and learners open song maps that conform to the shared, versioned JSON Schema. Later versions may add import or conversion tools if an earlier schema version or an external lesson format must be supported.

23.2 Data That Has to be Modified or Translated for the New System

No existing data must be converted for the initial release. Each lesson is authored directly in the song map format and validated before use.

If the schema's major version changes in a later release, a reader for the old version or a documented conversion process will be required so that existing valid maps remain usable (LO1). Translation, transliteration, timing, and annotation data are stored as explicit song map fields and do not need conversion in the initial release.

24 Costs

Development uses free tools and team members' devices. Table 8 lists costs that would apply only if the corresponding option is chosen.

The main cost is team time, planned in the Development Plan.

25 User Documentation and Training

25.1 User Documentation Requirements

- A learner guide covering importing a lesson, playback controls, display settings, sending words to Anki, and Anki setup for the selected handoff method (including the AnkiConnect settings in AC2, if used).
- A creator guide covering loading material, timing, latency calibration, reviewing segmentation, annotations, licence fields, and export errors.
- A description of the song map format for developers, generated from the JSON Schema.

25.2 Training Requirements

No training is required beyond the guides. Test creators in the G4 study receive the creator guide and the test material, and no other help (UH2).

Table 8: Possible costs

Item	When it applies	Cost
Apple Developer Program	Distributing the iOS app through TestFlight or the App Store	USD 99 per year [16]
Google Play developer account	Publishing on Google Play	USD 25 once [17]
Editor hosting	The editor is hosted on GitHub Pages	none
Licensed music	Using a commercial song in the acceptance suite	avoided by using songs created by the team or in the publi

26 Waiting Room

The following stretch goals from the Problem Statement are not required for the initial release:

- a third language and a pluggable approach to language processing,
- song map import and export tools (for example, LRC import),
- sharing and collaborative review of song maps,
- learner progress tracking, review inside the app, and further accessibility features,
- automatic timing and annotation assistance, with creator verification.

The following extensions of hazard mitigations are also future work. The risk they would reduce is accepted in the Hazard Analysis (Residual Risk):

- a listening mode designed for use while moving, beyond HS3,
- verified reviewer identities for IN5, which need user accounts,
- automatic delivery of error reports (AU1) to creators, which needs a backend,
- latency calibration that does not need the creator's help, beyond IN7.

27 Ideas for Solution

The proposed solution consists of a mobile learner app, a song map editor that runs in a web browser, and a shared TypeScript core. The learner app plays lesson audio, displays synchronized lyric lines, shows translations, transliterations, and annotations, and sends selected vocabulary to Anki. The editor creates and validates song maps without requiring creators to edit JSON files.

A versioned JSON Schema defines the song map, including metadata, timed lyric lines, words, translations, transliterations, and annotations. Using one validated format lets lessons created in the editor open in the learner app without changes.

The current implementation direction is React Native with Expo for the learner app, React for the editor, and TypeScript for the shared schema and validation logic. Japanese segmentation and romanization may use kuromoji.js in the editor. These are implementation ideas, not requirements. Other technologies may be used if the requirements and constraints are met.

28 Likely and Unlikely Changes

28.1 Likely Changes

LC1 The method for sending vocabulary to Anki (AnkiDroid API, AnkiConnect, or export file), depending on the PoC result for risk R3.

LC2 The second language, and later a third language.

LC3 The audio playback library in the learner app (Expo Audio or a native module), depending on the PoC result for risk R1.

LC4 The fields of the song map schema, as minor versions.

LC5 The Japanese tokenizer used by the editor (kuromoji.js or Sudachi).

LC6 The values of the symbolic constants in Appendix A, after measurement in the PoC and usability testing.

LC7 How song map packages reach the learner app (file import, bundled lessons, or a hosted catalogue).

28.2 Unlikely Changes

UC1 Learners use a phone app and creators use a web editor on a computer.

UC2 Lessons are song maps with timed lines, stored in one shared, versioned format.

UC3 The initial release has no user accounts and no server.

UC4 The target users are learners at early intermediate to intermediate levels who study on their own.

UC5 Anki is the spaced repetition system that receives vocabulary.

A Symbolic Constants

Values marked *initial* will be confirmed during the PoC or usability testing (LC6).

Constant	Value	Source or rationale
SYNC_TOL	0.5 s	Problem Statement, goal G1.
SYNC_PASS	95 %	Problem Statement, goal G1.
CREATE_TIME	45 min	Problem Statement, goal G4.
CREATE_PASS	2 of 3	Problem Statement, goal G4 (two of three test participants).
TEST_TRACK_LINES	12 lines	Problem Statement, goal G4.
LANG_COUNT	2	Problem Statement, goal G5.
LOAD_TIME	3 s	Development Plan, PoC pass criterion for R2.
SEG_ACC	90 %	Development Plan, PoC pass criterion for R2.
SEG_SAMPLE	20 lines	Development Plan, PoC pass criterion for R2.
SELECT_RESPONSE	0.5 s	Initial. Equal to SYNC_TOL, so the meaning appears no later than a late line change would.
ANKI_TIMEOUT	5 s	Initial. A local request normally completes in well under a second. 5 s allows for a slow device before reporting failure.
TIME_RES	0.01 s	One fiftieth of SYNC_TOL. Also the resolution of the LRC format.
AUDIO_DUR_TOL	1 s	Initial. Allows for differences in how encoders report duration.
DISCONNECT_PAUSE	1 s	Hazard Analysis, HS2.
MAX_FLASHES	3 per second	WCAG 2.1 Success Criterion 2.3.1 [8].
FADE_IN	0.5 s	Initial. Hazard Analysis, HS8. Long enough to avoid a sudden start, short enough not to hide the start of the first line.
TARGET_LOUDNESS	−16 LUFS	Initial. Hazard Analysis, HS9.
LOUDNESS_TOL	2 LU	Initial. Hazard Analysis, HS9.

Constant	Value	Source or rationale
AUTOSAVE_INT	60 s	Hazard Analysis, IN6. Limits lost work to one minute.
MAX_MAP_SIZE	5 MB (excluding audio)	Initial. A map at MAX_LESSON_LINES with annotations is estimated at well under 1 MB.
MAX_NESTING	32 levels	Initial. The schema nests fewer than 10 levels.
SUPPORTED_MAJOR	{1}	Major schema versions the current release reads.
MAX_TAPS_PLAY	3	Initial. Home screen, lesson, play.
LEARN_TIME	10 min	Initial. About the length of one study session.
MAX_AUDIO_LENGTH	10 min	Initial. Longer than most popular songs.
MAX_LESSON_LINES	150 lines	Initial. Covers a song of 10 minutes with short lines.
LIBRARY_SIZE	20 lessons	Initial. Several songs in progress, with storage dominated by audio files.
COVERAGE	80 %	Initial. Applies to the shared core only.

B Traceability

B.1 Goals to Requirements

Table 10: Goals and the requirements that refine them

Goal	Requirements
G1 Synchronized playback	FR3, FR4, FR5, FR10, FR13, LF1, LF2, UH1, SL1, PA1, PA2, RB2, CP1, IN1, IN7
G2 Support in context	FR7, FR12, LF2, UH5, SL2, CU2, IN5
G3 Interoperability	FR1, FR2, FR14, FR16, LO1, OE3, MS1, ST2, IN2, IN3, IN11, IM1, IM2
G4 Practical lesson creation	FR9 to FR15, UH2, CP1, IN6, IN7
G5 Writing system support	FR6, FR11, UH3, UH4, SL1, PA3, SC1, CU1, IN4, IN12
G6 Vocabulary handoff	FR8, UH5, SL3, OE2, AC1, AC2, PR2, IN8, IN9, IN10

B.2 Use Cases to Requirements

Table 11: Product use cases and functional requirements

Use case	Requirements
PUC1 Play synchronized lesson	FR1 to FR5
PUC2 Inspect a word or line	FR6, FR7
PUC3 Send a term to Anki	FR8
PUC4 Author a song map	FR9 to FR15

B.3 Hazards to Requirements

Table 12 lists, for each FMEA row of the Hazard Analysis, the requirements in this document that mitigate it.

Table 12: Hazard Analysis FMEA rows and mitigating requirements

Ref.	Failure mode	Requirements
H1.1	Highlighting drifts from the audio	IN1, IN2
H1.2	Audio starts unexpectedly or loudly	HS1, HS2, HS8, HS9
H1.3	Audio file missing or unplayable	IN11
H1.4	Learner watches the screen while moving	HS3, HS7
H2.1	Incorrect segmentation	IN4
H2.2	Incorrect transliteration	IN4
H2.3	Script does not render	IN12
H2.4	Rapid flashing	HS4
H2.5	Cues by colour only, fixed text size	HS5
H3.1	Incorrect translation or annotation	IN5, AU1, AU2
H3.2	Explicit content without warning	HS6
H3.3	Compulsive use or discouragement	HS7
H4.1	Map passes schema but is inconsistent	IN2
H4.2	Format version mismatch	IN3
H4.3	Crafted or malformed map	IM1
H4.4	Map refers to external URLs	IM2
H4.5	Map path points outside the package	IM2
H5.1	Creator loses unsaved work	IN6
H5.2	Creator overwrites timings	IN6, AU2
H5.3	Editor timestamps offset	IN7
H5.4	Unlicensed content	LG1
H6.1	Duplicate Anki notes	IN9
H6.2	Incorrect or garbled note fields	IN10
H6.3	App changes other Anki data	AC1
H6.4	Handoff fails silently	IN8
H6.5	AnkiConnect reachable by any web page or computer	AC2
H6.6	Unneeded permissions	PR2
H7.1	Queued vocabulary lost	IN8
H7.2	Data leaves the device	PR1, IM2
H7.3	Compromised or vulnerable library	IM3

References

[1] S. Robertson and J. Robertson, *Volere Requirements Specification Template*, Atlantic Systems Guild. https://www.volere.org/templates/volere-requirements-specification-template/

[2] Anki, *Anki Manual: Getting Started*. https://docs.ankiweb.net/getting-started.html

[3] AnkiDroid, *AnkiDroid API* (project wiki). https://github.com/ankidroid/Anki-Android/wiki/AnkiDroid-API

[4] Anki, *AnkiMobile Manual: URL Schemes*. https://docs.ankiweb.net/ankimobile/url-schemes

[5] A. Yatskov (FooSoft), *AnkiConnect* (add-on documentation). https://git.sr.ht/~foosoft/anki-connect

[6] World Wide Web Consortium, *Web Content Accessibility Guidelines (WCAG) 2.1*, W3C Recommendation. https://www.w3.org/TR/WCAG21/

[7] World Wide Web Consortium, *Understanding Success Criterion 1.4.3: Contrast (Minimum)*. https://www.w3.org/WAI/WCAG21/Understanding/contrast-minimum.html

[8] World Wide Web Consortium, *Understanding Success Criterion 2.3.1: Three Flashes or Below Threshold*. https://www.w3.org/WAI/WCAG21/Understanding/three-flashes-or-below-threshold.html

[9] World Wide Web Consortium, *Guidance on Applying WCAG 2 to Non-Web Information and Communications Technologies (WCAG2ICT)*. https://www.w3.org/TR/wcag2ict/

[10] JSON Schema, *Draft 2020-12*. https://json-schema.org/draft/2020-12

[11] T. Bray, Ed., *The JavaScript Object Notation (JSON) Data Interchange Format*, RFC 8259, IETF, December 2017. https://www.rfc-editor.org/rfc/rfc8259

[12] International Telecommunication Union, *Recommendation ITU-R BS.1770: Algorithms to Measure Audio Programme Loudness and True-Peak Audio Level*. https://www.itu.int/rec/R-REC-BS.1770

[13] LyricsTraining. https://lyricstraining.com/

[14] Musixmatch. https://www.musixmatch.com/

[15] Language Reactor. https://www.languagereactor.com/

[16] Apple, *Apple Developer Program*. https://developer.apple.com/programs/

[17] Google, *Get Started with Play Console*, Play Console Help. https://support.google.com/googleplay/android-developer/answer/6112435

All web sources were accessed on October 18, 2026.

C Appendix: Generative AI Use Disclosure

C.1 AI Tools Used

Table 13: AI tools used for this document

Tool	Version	Used for
OpenAI Codex desktop application	GPT 5.6 Terra model	Version 1.0 drafting
Claude Code (Anthropic)	2.1.158	Versions 1.1 to 1.4, inside VS Code with access to the repository
Claude Opus (Anthropic)	5.5 (`claude-opus-5-5`)	The model used by Claude Code

C.2 How and Where Used

Codex was used to draft and structure the Volere sections from the repository's project documents, in particular the functional requirements, business use cases, and data dictionary of Version 1.0.

Claude was used for Version 1.1. It was given this document, the Problem Statement, the Development Plan, the Hazard Analysis, the SRS checklist, and example requirements documents from previous capstone projects. It filled the remaining Volere sections, rewrote the requirements in the format of statement, rationale, fit criterion, and trace, added the Hazard Analysis requirements, and wrote the formal specification, likely and unlikely changes, symbolic constants, and traceability tables.

Claude was also used for Version 1.2 to address the review comments on Version 1.1: it drafted HS8 and HS9, the narrowed versions of HS3, IN5, IN7, and AU1, the revised AC2 and PR1, the description of the intended data flow under Compliance Requirements, and the References section.

On October 19, 2026, Claude was given the course grading rubric and used for Versions 1.3 and 1.4. It compared the document with each rubric row and drafted the intended audience, notation and conventions, the list of acronyms, the table of user characteristics, the third persona, and the phase in plan. It wrote a script that turns every requirement identifier in a trace into a link and adds the *Referenced by* lines, and it drafted the reflection answers from the commit history and review comments for the team to edit. A typical prompt was "review the SRS against this rubric and make the changes needed for full marks." Every change was made in a branch and reviewed as a diff in a pull request before merging.

C.3 Verification of AI-Generated Content

Content from Version 1.0 was checked by team members against project constraints and supervisor guidance:

- Verified that requirement fit criteria are measurable and align with the goals.
- Ensured that technical choices (React Native, TypeScript, kuromoji.js) are consistent across the repository's documents.

Content added with Claude in Versions 1.1 and 1.2 was checked as follows:

- **External claims:** the statements about Anki, AnkiDroid, AnkiMobile, and AnkiConnect, the WCAG, JSON Schema, RFC 8259, and ITU-R BS.1770 references, and the developer program fees were checked against the primary sources in the References section on October 18, 2026. This check corrected three statements: AnkiConnect does support an API key and listens only on `127.0.0.1` by default (AC2), AnkiMobile does provide a URL scheme for adding notes (Open Issues), and Anki organizes cards, not notes, into decks.
- **Product comparisons:** the descriptions of LyricsTraining, Musixmatch, and Language Reactor were checked against each product's website and store listings. Two were corrected: LyricsTraining

shows Japanese lyrics in rōmaji, and Language Reactor's paid version exports words for Anki.

- **Constants:** each value in Appendix A is either taken from the Problem Statement, the Development Plan, the Hazard Analysis, or a cited standard, or is marked *initial* with its reasoning, to be confirmed by measurement (LC6).
- **Consistency with the Hazard Analysis:** a script checked that every requirement traced to a hazard has the same identifier and FMEA rows in both documents, that Table 12 matches the FMEA table, and that every symbolic constant used is defined in Appendix A. The wording was also compared. It matches, except that numeric values are written here as symbolic constants.
- **Version 1.4 checks:** the trace script also checked that every one of the 85 requirements appears exactly once in Table 7 and that every identifier it links is defined. The document was compiled and the links, the table of contents, and the lists of tables and figures were checked in the PDF. The phase in dates were compared with the Development Plan.
- **Reflection:** the drafted answers are read by each team member, and anything that does not match their experience is rewritten in their own words before submission.

C.4 Limitations

AI was used to draft and check text, not to decide scope or priorities. The priorities and constant values are the team's decisions, and the values marked *initial* will be confirmed by measurement (LC6).

D Appendix: Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?
4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.
5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.
6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?